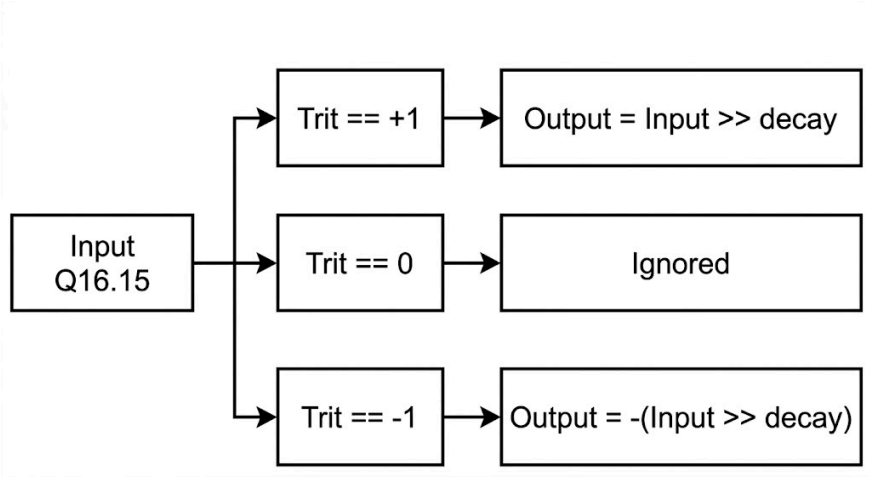
Quantized Adaptive Trainer Neural Network - Hybrid Fixed Point

1. Base Architecture and Forward Pass

This document presents **QATNN** (Quantized Adaptive Trainer Neural Network), an architecture based on the **Quantization-Aware Training paradigm** (QAT), distinguished however by the adoption of a **ternary regime governed by Shadow Weights** (Shadow Weights) and a **dynamic hysteresis map**.

- **Ternary Weights**: During inference, weights assume only values {-1, 0, +1}.
- **Inference and Vector Propagation**: As a property of ternary weights we can therefore replace heavy matrix computations with simple logical additions or subtractions.



***Implementation note**:* The parameter **decay** is specific to each layer (or channel) and is stored as a constant integer derived during training.

Global Scale Factor (γ_{shift}):

To restore the signal magnitude lost during the flattening of ternary quantization, the system uses a scale factor γ_{raw} specific to each layer (or channel). The accumulation of the absolute values of the Shadow Weights is performed through a **temporary 64-bit register** (`int64_t sum_abs = 0;`) to prevent **arithmetic overflow** before applying the bit-shift.

- **Architectural justification:** Empirical tests demonstrate that the scale factor is the load-bearing structural component of quantization: omitting this compensation (forcing $\gamma = 1$) severely compromises model optimization.

Computation of the Scale Factor (γ_{raw} and γ_{shift}): To eliminate the use of floating-point functions (**double**) and to guarantee native compatibility with the integer arithmetic of the ALU, the computation exploits bit-level hardware instructions (**__builtin_clz**) instead of **log2**:

$$\gamma_{raw} = (\Sigma|Shadow_i|) \gg (31 - _builtin_clz(N))$$

- Where N is the **fixed block size**, strictly equal to a power of two (e.g. 4096), guaranteeing the exact correspondence of the integer bit-level logarithm.
- The **global shift** factor is then derived as:

$$\gamma_{shift} = 31 - ((\gamma_{raw} == 0)?\, 31 : _builtin_clz(\gamma_{raw})) - 15$$

- **Final application (Pure Shift):** The layer concludes the accumulation without multiplications, scaling the raw output:

$$\text{If } \gamma_{shift} > 0 \Rightarrow Output_{shiftato} = Output_{grezzo} \ll \gamma_{shift}$$

$$\text{If } \gamma_{shift} < 0 \Rightarrow Output_{shiftato} = Output_{grezzo} \gg |\gamma_{shift}|$$

SiLUNS (SiLU Negative Shift): Division-Free Activation Function for QATNN Engines in Q16.15

To address the **architectural constraints of the Q16.15 fixed-point neural engine**—specifically the **prohibition on using floating-point arithmetic**, expensive exponential computations and hardware division circuits—we introduce **SiLUNS** (*SiLU Negative Shift*).

SiLUNS **approximates the mathematical properties** of the **SiLU** standard activation function ($f(x) = x \cdot \sigma(x)$), **mapping** its entire **behavior onto bit-shift operations** and piecewise-linear boundaries.

1. Mathematical Formulation and Structural Constraints

Operating within the **Q16.15**, the **dynamic safety range** intended for **SiLUNS is the interval [-2.0, +1.99997]**; although the format natively saturates at ± 65535, the rising branch requires this bound in order to **not cross zero toward an erroneous (positive)** output. The operating map is **divided across two main branches**:

- **Positive Pass ($x \geq 0$):**

  $$f(x) = x$$

  Signals preserve full fidelity, avoiding attenuation for strongly positive activations.

- **Negative "Pocket" Region ($0 > x \geq X_{min}$):**

  To replicate the characteristic **negative hump of SiLU by approximating a curved interior** and to guarantee absolute continuity (C^0) with no jumps that would destroy backpropagation, the **negative domain is split at the theoretical minimum point** ($X_{min} = -41900$, corresponding to ≈ -1.28). The joint is realized through two inverted-slope branches compensated by a constant algebraic offset:

  - **Descending Branch ($0 > x > -41900$):**

    $$f(x) = x \gg 2$$

  - The signal is gently attenuated until it reaches the **exact relative minimum** at the inflection point: $f(-41900) = -10475$ in **Q16.15** (about -0.319, an approximation of the true SiLU minimum at -0.278).
  - **Joined Rising Branch ($x \leq -41900$):**
    Beyond the minimum, the function must **rise back toward zero to dampen *negative outliers***. Instead of brutally zeroing the signal (causing a discontinuity jump), the slope is **inverted**

bitwise and an offset is applied to join the value perfectly to that of the previous branch:

$$f(x) =- (x \gg 2) - 20950$$

- ***Proof of continuity***:
  At the boundary $x =- 41900$, the equation returns $-(-10475) - 20950 = 10475 - 20950 =- 10475$, **joining the descending branch with no jump**. As x becomes more negative, the term $-(x \gg 2)$ becomes positive, smoothly **raising the function back toward zero** (e.g. at $x =- 65536$ the function equals -4566).

2. Hardware Efficiency and Resource Impact

By replacing transcendental exponential functions and **division circuits** with bit-level bit-shifts and conditional branches, **SiLUNS** guarantees **constant-time execution** ($O(1)$ ALU clock cycles), **preserving** at the same time **gradient flow during backpropagation** and eliminating any memory overhead.

Benchmarking and Performance Profiling: Validation tests on **SIMD vector inference** demonstrate that the **complete replacement** of **MAC instructions** (Multiply-Accumulate) **with conditional logical additions** (ternary weights) and bit-shift operations (SiLUNS and Gamma scaling) yields a **drastic reduction of computational cost**. This speedup is the combined result of eluding all physical multipliers on the ALU and of **aggressive packing at 1.625 bits per weight, which cuts memory-bandwidth latencies by 80%** (versus 8-bit) **or by 90%** (versus 16-bit) **memory bandwidth**.

2. Hysteresis Map for Trit Recomputation (Shadow Weight to Trit Mapping)

The conversion of Shadow Weights (Shadow in **Q16.15**) into ternary weights (**trit**) uses a **hysteresis** logic to **prevent frequent oscillations**:

- If Shadow >= T:
  - Previous trit = 1 -> New trit = 1
  - Previous trit = 0 -> New trit = 1
  - Previous trit = -1 -> New trit = 0
- If Shadow <= -T:
  - Previous trit = 1 -> New trit = 0
  - Previous trit = 0 -> New trit = -1
  - Previous trit = -1 -> New trit = -1
- If -T < Shadow < T:
  - Trit remains unchanged from the previous state

Initialization note: At model startup, the Shadow Weights (**Shadow**) are initialized with a controlled **Gaussian distribution** centered on zero. Unlike the later phases where **the hysteresis logic intervenes**, the ternary weights (**trit**) at state zero **have no "previous state"** on which to base the computation; therefore, **they too are initially allocated with random values** uniformly distributed over the set $\{-1, 0, +1\}$. This approach guarantees a **natively balanced statistical distribution**, breaking layer symmetry and providing the variance needed before the learning phase starts, then letting the network iteratively correct the values.

Stability note: The **Hysteresis Threshold** (**T**, defined by an optimized constant,213), operates **exclusively as a logical discriminant** for the state transition of the trits, allowing the Shadow Weights (**Shadow**) to evolve **freely in the continuous space** of the **Q16.15** format during the normal gradient-descent phases.
To preserve the numerical integrity of the 32-bit registers and **avoid destructive saturations** without blocking the learning dynamics, the **algebraic compensation**(by subtracting or adding the corresponding correction offset) is activated **exclusively in the vicinity of the critical structural limit** of the hosting register (**`INT32_MAX`**), which serves as the primary physical container to **prevent wrap-around**, well beyond the normal saturation margins of the mathematical **Q16.15**. This mechanism **guarantees that the weight remains continuous** and free of artificial constraints in its ordinary operating range, intervening only upon **reaching the extreme ceilings** to preserve the fixed-point remainders and **prevent hardware overflow**.

Zero-Trit Handling (Forward vs Backpropagation):

- **In Inference (Forward):** The conditional skip over null trits **is omitted** so as not to break CPU vectorization (SIMD). Processing **proceeds in continuous blocks through standard multiplication and addition operations**, regularly computing zero values as well **to keep the hardware pipeline saturated and match the efficiency of vector kernels**.
- **In Backpropagation (STE and Hysteresis Unlocking):** Trits equal to 0 must not **zero out the Shadow Weight gradient**. By applying the **Straight-Through Estimator** (STE), the pass is **treated as active (default value)**, preventing the Shadow Weight from remaining trapped at zero and allowing it to **accumulate variations so as to exit hysteresis**.

3. Physical File Structure and Memory Management (Data Locality Chunking)

To allow **dynamic and isolated loading** based on the **operating percentage** (%X), data are rigidly **partitioned into numbered "Chunks"**, where each file represents one layer (or **group of layers**). The system loads into RAM **only the strictly necessary chunks**.

Inference Files (I_X.bin Series)

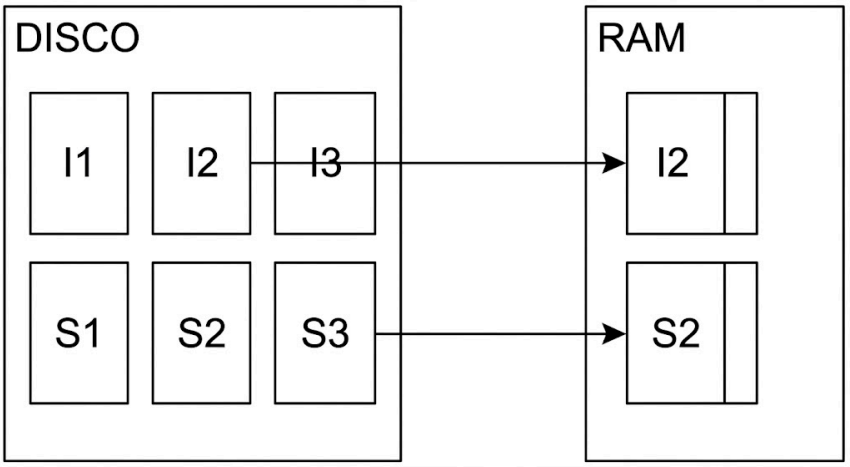
They contain the data for the Forward Pass and the inputs/outputs needed for Backpropagation for that specific block.

- **Naming**: I1.bin, I2.bin, I3.bin, etc.
- **Internal structure (contiguous)**: [Input] [Trit] [Output] [decay] [gamma_shift].
- **Ternary Weight Packing (Trit_Packed):** Values {-1, 0, 1} are compressed by grouping **128 weights into 26-byte blocks**. This yields an **effective in-RAM density of 1.625 bits per parameter** (leaving a margin of 2 unused trits per block **to favor memory alignment**). At 4096 by 4096 the trit field goes from **16.00 MiB to about 3.25 MiB**, reducing the footprint by almost **80%**.
- **Compatibility with the adaptive BITCOS format** (1.485 bits per weight) is activated **dynamically as the fraction of zeros learned** by the ternary weights during training grows (targeting a **structural sparsity** $\geq 51\%$, in line with recent standards in the ternary-model literature).
- **Loading**: When processing layer 2, only I2.bin is mmapped.

Optimizer Files (S_X.bin Series)

They contain the high-precision accumulators used **only for Backpropagation**, with a 1:1 correspondence to the inference files.

- **Naming**: S1.bin, S2.bin, S3.bin, etc.
- **Internal structure** (contiguous): [Shadow Weights][decay Weight].
- **Parallel Loading:** During backprop of layer 2, the system opens I2.bin and S2.bin. This selective loading brings into RAM **exclusively the necessary working set**, in line with the strategies of **optimized inference engines**, and **physiologically populates the operating-system page cache** to serve data on demand without saturating memory with the entire model.



Opening the third file in chunks:

In addition to the two expected files, it must also explicitly open the file **I_{X+1}.bin** of the next layer. This is mandatory to **fetch the transposed trit** required by the error formula (Error_X) of Section 5.
Structural exception: During backpropagation of the last layer (Layer L), this rule is suspended; the file $L + 1$ does not exist and is not requested, since the **initial error is injected directly by the loss function** with no backward propagation.

3.1 Asynchronous Prefetching and Double Buffering

To mask the physical latency of mechanical disks (HDD) and prevent bottlenecks while traversing layers, the architecture implements an **asynchronous I/O loading system** overlapped (**Double Buffering**). The system enforces a rigid hardware limit allowing at most **two layers simultaneously** (STREAM_MAX_RESIDENT = 2). While the **CPU compute threads process** the vector mathematics of the current block (X) in the first RAM slot, a **background thread pre-loads files** I_{X+1} **and** S_{X+1} into the second, idle slot. This mechanism guarantees a **smooth transition between layers, overlapping I/O times** with compute times without violating the host memory constraints.

4. Fixed-Point Formats and In-RAM Accumulators

All update arithmetic is performed through **integer formats**, rigidly defining the integer/fractional bit split, **supported by temporary accumulators non-persistent** resident in RAM during the 8-inference cycle:

1. **Shadow (**int32_t**):** Format **Q16.15** (1 sign bit, 16 integer bits, 15 fractional bits). The choice of **32 bits guarantees that overflow is almost impossible** yet still extremely controlled, **at the cost of losing 50% of the values**. The Hysteresis Threshold **T** is fixed at 213. It operates as a logical discriminant; the update step (Section 7) scales this constraint, and for a typical $Update_{Gain} = 4$ this is equivalent to a minimum accumulation of $\Delta = 256$ to trigger.
2. **Output (**int32_t**):** Temporary accumulator in **Q16.15** (1 sign bit, 16 integer bits, 15 fractional bits), **initialized to 0** for each neuron/channel at the start of the Forward Pass and **iteratively updated through standard operations** of multiplication and accumulation (SIMD):

   $$Output_{grezzo} \mathrel{+}= (Trit_{ij} \times (Input \gg decay))$$

   At the end of the layer computation, the **shifted value** is generated by applying the **global scale factor**:

   $$\text{If } \gamma_{shift} > 0 \Rightarrow Output_{shiftato} = Output_{grezzo} \ll \gamma_{shift}$$
   $$\text{If } \gamma_{shift} < 0 \Rightarrow Output_{shiftato} = Output_{grezzo} \gg |\gamma_{shift}|$$

   To which **the** SiLUNS activation function described above is subsequently applied to obtain the **final value**.

   $$Output_{finale} = SiLUNS(Output_{shiftato})$$

3. **Gradient (**int16_t**):** Format **Q4.11** (1 sign bit, 4 integer bits, 11 fractional bits).
4. **Learning Rate / LR (**uint16_t**):** Format **UQ0.16** (0 sign bits, 0 integer bits, 16 fractional bits).
5. **Absolute Error Accumulator Acc_Err(**uint32_t**): UQ7.25** (0 sign bits, 7 integer bits, 25 fractional bits) RAM variable for iteratively summing the absolute values of the errors recorded in inferences 1–7.
6. **Gradient Accumulator Acc_Grad(**int64_t**):** RAM variable for iteratively summing the raw gradients over the same time interval as the absolute-error accumulator.
7. **Gamma-Shift Scale Factor (γ_{shift}) (int8_t):** A signed 8-bit integer (sufficient to hold the standard hardware shift range from -31 to $+31$). It directly stores the operand

to be passed to the output bit-shift instruction at the end of the forward pass, occupying only 1 byte.

4.1 HFP Numerical Contract (Hybrid Fixed Point)

Despite the architectural prohibition on using floating-point multipliers for weight matrix computation (replaced entirely by logical additions and bit-shifts), **the architecture defines itself as Hybrid (HFP)** in that it **confines the use of floating point** (FP32/Double) **exclusively to vector-stabilization operations**, external to the primary inference engine. Specifically, the numerical contract of the system imposes the following division:

Integer/Fixed-Point Domain (Zero FPU): Shadow Weights **(Q16.15)**, Ternary weights **(trit)**, Gradients **(Q4.11)**, Decay parameters **(UQ4.12)** and Global Scale γ_{shift} **(int8_t)**.
The entire matrix traversal of the layers (forward and backward pass) takes place **without the aid of floating-point instructions**.

Floating-Point Domain (FP32/Double): FPU usage is **strictly limited to normalizations (RMSNorm)**, to maintaining the **temporal recurrent state** (RWKV format) and to the terminal computation of the **loss function** (Softmax/Cross-Entropy). This spatial hybridization is an indispensable mathematical compromise: it **preserves O(1) acceleration** and the very low RAM consumption of ternary weights for **95%** of the computations, exploiting floating point only in the **nonlinear routing functions** to prevent **numerical collapse of the gradient**.

4.2 Language Model Integration (RWKV Architecture)

Unlike the standard Transformer architecture, the **QATNN neural engine makes no use of the multi-head causal Attention mechanism** (Multi-Head Attention) **nor** of the expensive **spatial Softmax matrix operation**. Such operations would in fact require **quadratic computational complexity**, incompatible with the hardware constraints of a fixed-point engine for CPU. The model **instead implements the RWKV-6 (Finch) linear recurrence**, combined with a **Pre-Norm structure and a SwiGLU-type MLP block** ($ff = SiLUNS(W_g \cdot x) \cdot (W_u \cdot x)$). Text processing is performed through a **BPE (Byte Pair Encoding) vocabulary** integrated with a **ternary Embedding matrix**. By exploiting the **temporal memory** typical of **RNNs** (computed in linear time and row by row), the system completely eliminates Attention-related bottlenecks, allowing the ternary engine to **process complex documents at constant resources**.

5. Error Propagation and Gradient Computation (Layer Lookup)

In the block system with separate files, backpropagation **does not keep everything in RAM** but proceeds backward **by loading the files of the current group** X. Exploiting the data saved in the forward pass, **computation proceeds through the following algebraic relations**:

1. **Error Initialization on the Last Layer (**Output**):**
   Backpropagation starts from the last layer of the network (layer L). The initial error is computed as the **direct difference** between the **predicted output** by the model and the **target (expected value) in the integer reference format:**

$$Error_L = Output_{predetto} - Target_{atteso}$$

$$Loss = \Sigma_{k=1\ldots N}(Output_k - Target_k)^2 \gg 19$$

   Architectural note: The $\gg 19$ quantizes **exclusively** the reported loss value for printing, not the differentiated function. The derivative entering the gradient computation G_L is simply: $\frac{\partial Loss}{\partial Output_L} = 2(Output_L - Target_L)$ in **Q16.15** (with no shift).

   The only 19-bit shift applied for the **last layer** is that of the **outer product** described in point 3, indispensable for converting the accumulation from **Q32.30** to **Q4.11**.

2. **Error Computation for the Current Group (X):**
   To propagate the error toward the start of the network, the error received from the next group ($X + 1$) is multiplied by the **transposed Trits of the next group ($X + 1$)**:

$$Error_X = Trit^T_{X+1} \times Error_{X+1} \ (Result:\ a\ vector)$$

3. **Gradient Computation for the Current-Group Weights (X):**
   To determine the correction for the current-block weights, the **next-group error** received is taken and multiplied by the **current-group Input (X)** (coinciding with the saved output of the current group X):

$$G_X = (Error_{X+1} \otimes Input_X) \gg 19$$

$$(Logical\ result:\ outer\ product\ in\ Q4.11\ format)$$

   To **prevent peak errors from causing a destructive overflow** during vector propagation, the result **is forced preventively** (clipped) into the **native saturation field of the Q4.11 format**, namely the strict interval $[-2047, +2047]$.

   Architectural note on allocation (on-the-fly computation): Since the gradient for a single token derives from a rank-1 outer product, the **matrix G_X is never**

materialized nor wholly allocated in RAM. Processing proceeds strictly row by row: the gradient is computed, instantly applied to update the Shadow Weight, and immediately discarded. This optimization **cuts the memory requirement for the gradient from 26 GB to fractions of a Megabyte: 26 GB is 2 bytes per parameter, hence it holds only for models with more than 13 billion parameters. On a 7-billion model the required estimate is 14 GB, not 26**, guaranteeing the feasibility of block-wise backpropagation.

Format-alignment note: Since the outer product between error and input (both in **Q16.15**) natively generates an integer accumulation in **Q32.30**, a **right shift by 19 bits (**$\gg 19$**)** is applied to rigorously convert the datum into the **Q4.11** gradient format. This algebraic truncation **prevents destructive saturation** of the registers and prevents self-shutdown of the optimizer.

Safety and Accumulation Constraint: The accumulation of the outer product between error and input is temporarily performed in a **64-bit register** (`int64_t`) to **preserve the full precision of the Q32.30 format** before applying the 19-bit right shift. This approach removes any artificial restriction on the input field, guaranteeing compatibility with the entire interval representable by the **Q16.15** format with no destructive saturations.

4. **Absolute Error Accumulator Acc_Err(**uint32_t**):** Support variable **not persisted to file** (RAM-resident) used to **sum the absolute values of the errors during the 7 preliminary inferences**, zeroed at the 8th cycle after the Learning Rate computation.

6. Decay Optimization (UQ4) and Support-Weight Management

The decay parameter (decay) governing the bit-shift is not static, but is iteratively optimized during backpropagation through a high-precision support value.

Decay Shadow: Stored as an unsigned 16-bit integer in **UQ4.12** (4 integer bits, 12 fractional bits).

Initialization:

$$decay_{Weight} = min(15, round(0.5 \times log_2(N_{in}))) \times 2^{12}$$

- truncated to the **UQ4.12 field**. The initial value is the one bringing the accumulator $Output_{grezzo}$ back near 1: starting from 0 multiplies the accumulator by about 1000 and drives γ_{shift} out of range.

  N_{in} is the length of the **reduction axis of the dot product** in the forward pass, per output channel: it indicates **how many inputs converge onto each output**.

- **G_decay** = It is obtained by computing the **error with respect to the decay parameter**. In practice, during the backward pass (**backprop**), one measures how much the current error is affected by the present shift value:

  $$G_{decay} = ((\Sigma\, |Error_n \times Input_n|) \gg 8)\ \&\ 0xFFFF$$

  (The accumulation is shifted and masked to 16 bits to prevent overflow before the parameter update)

  This gradient indicates the **direction and intensity with which the bit-shift must be corrected** (increasing or decreasing it) to improve signal attenuation.

- **Update and Quantization Process (Cut):**
  1. **Delta_decay** = (G_decay * LR) >> 15
  2. **decay_Weight** = decay_Weight - Delta_decay
  3. **Extraction (Cut):** For the next inference, the fractional part is truncated to extract the pure integer destined for the bit-shift operation: decay = decay_Weight >> 12 *(The final value is clipped to a safe range, e.g. 0-15).*

7. Backpropagation and Weight Update

The correction computation exploits bit-shifts to avoid overflow and align formats, discarding excess fractional bits.

1. $decay_{Weight} = \min(15, (31 - _builtin_clz(N_{in}))\gg 1)\ll 12$

   Architectural note: Update_Gain *is an integer with* ***0 <= Update_Gain <= 12****, reconciling* ***the limits of the UQ0.16*** *of the* ***Learning Rate*** *(saturation at 65535) with the high hysteresis threshold (2^{13}). By reducing the right shift, it* ***allows real gradients to accumulate sufficient magnitude to unlock the trit state*** *without requiring 32-bit data formats for the LR.*
2. **Shadow** -= Delta_Shadow

7.1 Structural Mechanisms of Convergence and Stability

For the ternary model to overcome the initial stall phase and to keep gradients from **undergoing destructive saturations**, the architecture **implements four convergence levers** that operate in synergy during the weight update:

Gradient Rescaling (Grad-Scale): The raw outer product, **computed in Q32.30** and shifted **right by 19 bits**, generates a magnitude **intrinsically too large** for the **destination Q4.11 format**. The system therefore applies an **attenuation mechanism** based on an **8-step moving average** combined with a hard block (**clamping**), keeping updates within a **safe tolerance range** and preventing gradient explosion.

Gain Calibration (Update_Gain): In the computation of the Delta for the Shadow Weights ($\Delta_{Shadow} = (G \times LR)\gg(12 - Update_Gain)$), the parameter $Update_Gain$ is **set to 0 by default**. This precise mathematical configuration allows the equation to **accumulate the exact inertia** needed to **overcome the Hysteresis Threshold** ($T = 213$), while eluding the structural saturation of the **Learning Rate in the UQ0.16 format**.

Initial Gaussian Noise (Sigma): At startup, the Shadow Weights **are not initialized near zero**. Instead they are **forced through a Gaussian distribution with standard deviation** $\sigma = 2048$ (about 10 times the hysteresis threshold). This extreme asymmetric noise is vital: it **breaks model symmetry** and assigns an initial direction to the weights **even before the network processes the first gradient**.

Warmup and Stall-Halving (WSD): Training exploits a WSD curve **(Warmup-Stable-Decay)**. The first 10 computation steps **serve as warmup to stabilize the derived gradients**, implementing **in parallel** a **dynamic halving of the learning step** (Stall-Halving) whenever the loss function registers a prolonged stall.

8. Adaptive 8-Inference Cycle (%X and LR)

The system evaluates and updates the learning state in blocks of 8 inferences to guarantee stability, completely avoiding divisions in favor of bit-shifts:

- **Phase A (Inferences 1–7):** Incremental collection and parallel accumulation of the absolute values of errors and of raw gradients in the RAM accumulators.
- **Phase B (Inference 8)**:
  - **Mean Gradient Approximation (G_{avg}):** Since exact division by 8 requires costly divisions, the bitwise approximation for division by 8 or 16 is used (e.g. by approximating the window to 8 or 16 steps, or by multiplying by the fixed-point reciprocal). Using an 8-inference window, the computation becomes a simple 3-bit right shift:
    $$G_{Avg} = (Acc_{Grad} + G_{Curr}) \gg 3$$
  - **Preventive Noise Clipping:** Preventive Clipping does not operate on Acc_Err: it is an **unsigned accumulator that saturates by construction** and its field is already limited to **UQ7.25**. The only quantity requiring clipping is the raw gradient G, and it is **already constrained to [-2047, +2047]** by Section 5.

    Learning Rate (LR) Computation and Explicit Cooling:
    To **prevent** the **optimizer from prematurely converging** onto a **fixed point** (blocking learning), the update removes the **self-referential inertia** and adopts an **explicit decay mechanism driven** exclusively by the residual error. The computation applies an *annealing schedule* (Simulated Annealing) by dynamically incrementing the divisor through bit-shifts:

    $$LR_{new} = max(64, min(65535, (|G_{Avg}|) + (Acc_{Err} \gg (9 + Cooling_{Shift}))))$$

    The explicit use of parentheses in the exponent **(9 + Cooling_Shift)** guarantees, at implementation level, that cooling **acts strictly as a growing divisor**, compressing the learning step instead of adding to it.

    Asymptotic Decay (Cooling_Shift**):** It is an integer parameter (initialized to 0) that is incremented gradually (e.g. every 1024 macro-epochs). As **training progresses**, the division goes from $\gg 9$ to $\gg 10, \gg 11$, actively compressing the learning step as the network approaches the global minimum.
    Floor Clamp (64): The **max** operator enforces the structural limit in the **UQ0.16**. When the error collapses or **cooling becomes extreme**, the parameter rests on the value 64, guaranteeing room for terminal **micro-adjustments** (fine-tuning) without ever causing shutdown by mathematical collapse to zero.

9. Adaptive Depth Management (Layer Cut)

The computation of the number of layer files to load from the percentage **%X** uses the native integer arithmetic of the C language:

$$\min\left(totlayer, \frac{\min\left(\max\left(40, totlayer\right), 42949672\right) \times p}{100}\right) \text{-> uint32_t}$$

- **Role of parameter 40:** The value **40** acts as a **minimum-threshold operand** in the **multiplication factor for reduced percentages on compact models**, preventing the **file count from prematurely dropping to zero**. In networks with fewer than 40 layers, the outer **min** correctly returns the effective total (**totlayer**)
- **Integer truncation:** Integer CPU division natively truncates the fractional part without requiring math libraries. **Precision loss is confined to the algebraic truncation of the percentage**, without impacting the stability of selective loading.

10. References

1. BitNet & BitLinear (Ternary Weights)

- **The Era of 1-bit LLMs (BitNet b1.58):** arXiv:2402.17764
- **BitNet b1.58 2B4T Technical Report:** arXiv:2504.12285
- **Official Microsoft BitNet Repository:** GitHub - Microsoft/BitNet

2. RWKV Architecture

- **RWKV: Reinventing RNNs for the Transformer Era:** arXiv:2305.13048
- **Official RWKV-LM Repository:** GitHub - BlinkDL/RWKV-LM

3. Quantization and Fixed-Point Formats (Q-Format)

- **Quantizing the RWKV Family (PTQ):** OpenReview - RWKVQuant
- **Fixed-Point Arithmetic & Scaling in Neural Networks:** KIT Publications - Fixed-Point Quantization
- **Jacob, Kligys, Chen et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference"**, CVPR 2018 (arXiv:1712.05877). The present model departs from this theoretical basis by replacing pure integer inference with a ternary regime governed by Shadow Weights and a dynamic hysteresis map.

4. Advanced Ternary Literature (2026)

- **Sparse-BitNet**: *Sparse-BitNet: Scaling 1-bit LLMs with Structured Sparsity*, arXiv:2603.05168 (2026).
- **TernaryLM**: *TernaryLM: Native Ternary Large Language Models*, arXiv:2602.07374 (2026).

5. Character-level BPE Vocabulary Used by the Engine for Text Processing

- **"Neural Machine Translation of Rare Words with Subword Units" (2015):** arXiv:1508.07909 Sennrich, R., Haddow, B., & Birch, A.

6. Reference for the Original SiLU Activation Function

- **"Sigmoid-weighted linear units for neural network function approximation in reinforcement learning" (2017):** arXiv:1702.03118 Elfwing, S., Uchibe, E., & Doya, K.

7. Application of the standard I/O overlap paradigms (Asynchronous I/O Overlap) derived from high-performance computing (HPC) architectures, adapted to MAX_RESIDENT memory constraints.

Calculations Powered By:

Gemini 3.1 Pro-Extended,
Space Bunny 1.0 [Freebuff],
Muse Spark 1.3 [Freebuff],
My Own Project Ideas and Inputs